

CS F301 POPL Assignment: Functional-OO File Processor Date

Manas Choudhary (2022B3A70784G), Rahul Kumar Yadav (2023A7PS0341G), Tanishq Gupta (2023A7PS0337G)

1. Interpretation of Specifications

We interpreted the requirement of "Functional-OO" as designing a system where the WHAT (the desired data transformations) is specified using high-level abstractions, while the HOW (loops, state management) is abstracted away using functional paradigms like Lambdas and Streams. We focused on creating a pipeline that Reads -> Transforms -> Aggregates -> Sorts/Searches.

2. Chronological Design and Development

- **[10:00 AM]:** Initial analysis of the assignment PDF. We identified the need for a non-procedural approach.
- **[11:30 AM]:** We designed the "WHAT" interface. We decided on a menu-driven CLI for ease of testing.
- **[01:00 PM]:** Rahul implemented the `FileReaderAPI` to handle IO streams functionally.
- **[02:45 PM]:** Tanishq implemented the `AggregationUtils` to handle "counting inversions" and "averages" as requested in the spec.
- **[04:00 PM]:** Manas integrated all modules into `FunctionalFileProcessor` and performed testing with dummy data.

3. Usage of GenAI Tools

As permitted, we utilized LLMs primarily for minor syntactic formatting and to improve the clarity of code comments and documentation. The architectural design and logic were derived manually.

- **Prompt 1:** "Refactor these comments to be more concise and check the indentation of this lambda function for standard formatting."
 - **Outcome:** The AI assisted in cleaning up the whitespace and ensuring the inline comments were grammatically consistent, improving code readability.
- **Prompt 2:** "Suggest better variable naming conventions for this specific filtration method to make it more self-explanatory."
 - **Outcome:** We adopted clearer naming conventions for the helper functions to ensure the code self-documents the "WHAT" aspect effectively.

4. Features

- Menu-driven interface.
- Functional keyword search (Map/Reduce).
- Mathematical aggregations (Inversions, Averages) on unstructured text.
- Modularity: IO, Logic, and Math are in separate classes.

Github Link: <https://github.com/Tannu-Git/POP>